Dome - Dome Core EVM SDK

Audit Report

Version 1.1

Zigtur

April 24, 2026

Dome - Dome Core EVM SDK - Audit Report

Zigtur

April 24, 2026

Prepared by: Zigtur

Table of Contents

Introduction

Disclaimer

A smart contract security review cannot guarantee the complete absence of vulnerabilities. This effort, bound by time, resources, and expertise, aims to identify as many security issues as possible. However, there is no assurance of 100% security post-review, nor is there a guarantee that the review will uncover all potential problems in the smart contracts. It is highly recommended to conduct subsequent security reviews, implement bug bounty programs, and perform on-chain monitoring.

About Zigtur

Zigtur is an independent blockchain security researcher dedicated to enhancing the security of the blockchain ecosystem. With a history of identifying numerous security vulnerabilities across various protocols in public audit contests and private audits, **Zigtur** strives to contribute to the safety and reliability of blockchain projects through meticulous security research and reviews. Explore previous work here or reach out on X @zigtur.

About Dome

Dome is a decentralized protocol for secure and anonymous transactions built on Solana. It is now expanding to EVM chains.

Using zero-knowledge proofs, it lets users deposit assets and withdraw them to different addresses, breaking the link between sender and receiver. By combining advanced cryptography with decentralized infrastructure, Dome restores financial privacy and freedom on public blockchains.

Security Assessment Summary

Review commit hash - 3dec950

Fixes review commit hash - 97e58c7

Scope

- src/*

Deployment chains

- Base

Risk Classification

	Impact: High	**Impact:** Medium	**Impact:** Low
Likelihood: High	High	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Issues

MEDIUM-01 - UTXO private keys written to plaintext temporary file in Node.js proving path

Scope:

- src/utils/prover.ts#L54-L65

Description

When the SDK detects it is running in a Node.js environment (i.e., not a browser), it generates ZK proofs by invoking the `snarkjs` CLI as a subprocess. The circuit inputs are written to a temporary file before being passed to the subprocess:

```
const tmpDir = await fs.mkdtemp(path.join(os.tmpdir(), 'dome-proof-'));
const inputPath = path.join(tmpDir, 'input.json');
// ...
await fs.writeFile(inputPath, JSON.stringify(utils.stringifyBigInts(input)));
```

The `input` object passed to the circuit contains highly sensitive fields constructed in `getProof` (`utils.ts`):

```
inPrivateKey: inputs.map((x) => x.keypair.privkey.toString()),
```

The `privkey` is the UTXO private key that authorizes spending. It is derived directly from the user's wallet signature. Writing this to disk exposes it to:

1. **Other processes running as the same OS user**: The temporary directory has mode `0700`, restricting access to the creating user. However, any other process running with the same user ID (e.g., another server process, a compromised dependency, a debugger) can read the file.
2. **Disk persistence on abnormal termination**: The cleanup is in a `finally` block, which handles most cases. However, a `SIGKILL` signal or system crash prevents cleanup, leaving the private key on disk indefinitely.
3. **Swap/hibernation disclosure**: OS swap files or hibernation images can capture in-memory or file-system content, including temp files, and may persist after rebooting.

Recommendation

- Prefer the in-memory browser path (`proveBrowser`) over the CLI subprocess path whenever possible.
- For Node.js environments, use `snarkjs` programmatically (via `require('snarkjs').groth16.fullProve(...)`) rather than spawning a subprocess, eliminating the need to write inputs to disk.

- If the CLI path must be used, immediately overwrite the `input.json` file with zeros before deletion to prevent recovery.

Dome

Fixed in PR3.

Zigtur

Fixed. Files are not used anymore, snarkjs library is directly used.

LOW-01 - Privacy leaks and trust dependency on centralized indexer

Scope:

- src/utils/utils.ts#L113-L141
- src/utils/utils.ts#L279-L308

Description

The SDK routes all critical operations through a centralized indexer without on-chain verification. This creates both privacy leaks and a single point of trust that undermines the protocol's decentralization guarantees.

Privacy leaks:

Every SDK operation discloses user activity to the indexer. When spending a UTXO, the SDK posts the commitment hash to `POST /commitment/`:

```
let res = await fetch(`${INDEXER_URL}/commitment/`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ commitment: toFixedHex(input.getCommitment()) }),
});
```

This reveals exactly which UTXO the user intends to spend. Combined with IP-level metadata and the timing of `GET /merkle/root` requests, the indexer can correlate spending patterns to specific users. Similarly, `GET /get_encrypted?start=...&end=...` calls during UTXO discovery expose scanning activity, allowing the indexer to infer which address ranges a user is interested in and when they check their balance.

In aggregate, the indexer can build detailed activity profiles: deposit times (via `/screen_address`), spending patterns (via `/commitment/`), and balance-check frequency (via `/get_encrypted`). This effectively deanonymizes users despite the on-chain privacy provided by the ZK proofs.

Trust dependency:

The Merkle root, output UTXO indices, and Merkle path elements are all fetched from the indexer and used directly without on-chain verification:

```
const res = await fetch(`${INDEXER_URL}/merkle/root`);
const { root, nextIndex: initialNextIndex } = await res.json();
```

A compromised or malfunctioning indexer can return an invalid Merkle root (causing proof generation to waste computation then fail on-chain), an incorrect `nextIndex` (causing output UTXOs to be encrypted with a wrong index, making them unrecoverable), or wrong path elements (selectively denying service to targeted users). While the on-chain verifier prevents fund theft — invalid proofs are rejected — the

indexer retains the ability to cause denial of service and permanent fund locking through incorrect `nextIndex` values.

Recommendation

- Route indexer requests through a privacy-preserving layer (e.g., Tor) or use a local indexer node to prevent IP-based activity correlation.
- Before generating a proof, verify the Merkle root on-chain via `etherPool.isKnownRoot(root)`.
- After a successful transaction, verify the assigned output indices by reading `etherPool.nextIndex()` from the chain.
- Provide a fallback that reconstructs Merkle paths directly from on-chain `NewCommitment` events when the indexer is unavailable or suspected of misbehavior.

Dome

Acknowledged. There's no logging on commitment path. Also querying commitment is a separate call from transact call, indexer can't mathematically guarantee whoever fetch the commitment is the one making the next transaction.

Zigtur

Acknowledged.

INFO-01 - Dead `relayer` parameter in SDK `getExtDataHash` creates code inconsistency

Scope:

- src/utils/utils.ts#L53-L89

Description

The SDK's `getExtDataHash` function accepts a `relayer` parameter that is never used in the ABI encoding:

```
export function getExtDataHash({
    recipient,
    extAmount,
    relayer,          // accepted but never used
    feeRecipient,
    fee,
    encryptedOutput1,
    encryptedOutput2,
}: {
    ...
    relayer?: any;    // optional — silently ignored
    ...
})
```

The ABI encoding does not include `relayer`, and neither does the on-chain `keccak256(abi.encode(_extData))` computation (which encodes `recipient, extAmount, feeRecipient, fee, encryptedOutput1, encryptedOutput`
This parameter is dead code inherited from an earlier version.

The core `src/utils.js` in the companion repository does not have a `relayer` parameter in its `getExtDataHash`, making these two implementations inconsistent.

Recommendation

Remove the `relayer` parameter from `getExtDataHash` to eliminate dead code and align the SDK's implementation with the core utilities.

Dome

Fixed in PR3.

Zigtur

Fixed.

INFO-02 - No on-chain event fallback for UTXO discovery if indexer is unavailable

Scope:

- src/utils/utils.ts#L235-L337

Description

The `findUnspentUtxos` function relies exclusively on the centralized indexer API (`GET /get_encrypted`) to retrieve encrypted output data. If the indexer is unavailable (downtime, network issue, or deliberate denial of service), users are completely unable to discover their UTXOs and therefore cannot spend or verify their funds:

```
let url = `${INDEXER_URL}/get_encrypted?start=${startIndex}&end=${startIndex +
↪  limit}`;
const res = await fetch(url);
if (!res.ok) {
    throw new Error(`Failed to fetch encrypted UTXOs: ${res.statusText}`);
}
```

The `EtherPool` contract emits `NewCommitment` events containing the full `encryptedOutput` bytes for every transaction. This on-chain data is sufficient to reconstruct a complete UTXO set independently of the indexer, but the SDK provides no fallback to query these events directly.

Recommendation

Implement a fallback UTXO discovery mode that queries `NewCommitment` events directly from the RPC provider (e.g., using `etherPool.queryFilter(etherPool.filters.NewCommitment())`) when the indexer is unreachable. This ensures users retain access to their funds even if the indexer is offline.

Dome

Acknowledged. We will decentralize relayers one day to add service redundancy.

Zigtur

Acknowledged.

INFO-03 - `feeRecipient` encoding inconsistency between SDK and core utils

Scope:

- src/utils/utils.ts#L82

Description

In the core repository's `src/utils.js`, `feeRecipient` is encoded using `toFixedHex(feeRecipient, 20)` before ABI encoding:

```
// core/src/utils.js
feeRecipient: toFixedHex(feeRecipient, 20),
```

In the SDK's `src/utils/utils.ts`, `feeRecipient` is passed directly without the `toFixedHex` call:

```
// sdk/src/utils/utils.ts
feeRecipient,   // raw value, no toFixedHex
```

In practice, when `feeRecipient` is a well-formed checksummed Ethereum address string (e.g., `"0x44eb..."`), ethers.js ABI encodes both forms identically for the `address` type (left-padded to 32 bytes). The resulting `extDataHash` is therefore the same in both cases under normal operation.

However, the inconsistency introduces risk: if `feeRecipient` is passed as a value other than a standard address string (e.g., a BigNumber, a buffer, or an un-checksummed address), the two implementations could diverge and produce different hashes — causing the on-chain extDataHash check to fail.

Recommendation

Apply `toFixedHex(feeRecipient, 20)` to the `feeRecipient` field in the SDK's `getExtDataHash` to match the core implementation and prevent encoding divergence.

Dome

Acknowledged.

Zigtur

Acknowledged.